SDD — Visualizador de Estructuras de Datos Secuenciales en C estándar con Raylib

Proyecto: Visualizador académico de estructuras de datos secuenciales
Módulo 1: Pilas, colas, colas de prioridad y listas enlazadas
Lenguaje: C estándar
Biblioteca gráfica: `raylib.h`
Contexto académico: Asignatura Estructuras de Datos — Ingeniería de Sistemas y Computación — Universidad Tecnológica de Pereira
Versión: 1.0
Autor académico: Francisco Alejandro Medina Aguirre

---

1. Información general del proyecto

1.1 Nombre del sistema

Visualizador de Estructuras de Datos Secuenciales en C estándar con Raylib.

1.2 Descripción general

El sistema será una aplicación de escritorio desarrollada en **C estándar**, con interfaz gráfica construida mediante **Raylib**, cuyo propósito es apoyar el aprendizaje de estructuras de datos secuenciales. El visualizador permitirá al estudiante observar de forma simultánea:

1. La representación gráfica de una estructura de datos.
2. El estado interno de la estructura después de cada operación.
3. El código asociado a la operación que se está ejecutando.
4. La explicación algorítmica paso a paso.
5. La complejidad temporal y espacial de cada operación.

El primer módulo estará enfocado en las estructuras secuenciales estudiadas en el curso:

- Pila.
- Cola FIFO.
- Cola de prioridad.
- Lista enlazada simple.

El sistema debe integrarse con los TAD suministrados en los archivos:

- `pila.h` / `pila.c`
- `cola.h` / `cola.c`
- `cola_prioridad.h` / `cola_prioridad.c`
- `lista.h` / `lista.c`

También se tendrán en cuenta los módulos gráficos iniciales suministrados:

- `pilas_raylib.c`

- `colas_raylib.c`
- `colas_prioridad_raylib.c`
- `listas_raylib.c`
- `ui.h`
- `ui.c`
- `main.c`

1.3 Objetivo del sistema

Diseñar e implementar una herramienta visual e interactiva que permita complementar el análisis algorítmico de las estructuras de datos secuenciales mediante una representación gráfica, modular y documentada, usando C estándar, Raylib y TAD separados por interfaz e implementación.

1.4 Alcance del módulo 1

El módulo 1 incluirá la visualización y operación básica de:

Estructura	Operaciones principales
Pila	Inicializar, push, pop, contar, vaciar/destruir, mostrar
Cola	Inicializar, encolar, desencolar, contar, vaciar, mostrar
Cola de prioridad	Inicializar, encolar con prioridad, desencolar por prioridad, contar, vaciar, mostrar
Lista enlazada	Insertar inicio, insertar final, insertar antes, insertar después, buscar, eliminar primero, eliminar todos, invertir, promedio, mayor, verificar orden

No se incluirán todavía árboles, tablas hash ni grafos. Estos podrán diseñarse como módulos posteriores.

---

2. Visión del sistema

2.1 Problema que resuelve

En el aprendizaje de estructuras de datos en C, muchos estudiantes comprenden la sintaxis de las funciones, pero presentan dificultades para visualizar:

- Cómo cambian los punteros después de cada operación.
- Qué nodo se crea o elimina.
- Qué elemento corresponde al tope, frente, final o cabeza.
- Por qué ciertas operaciones son `O(1)` y otras `O(n)`.
- Cómo se relaciona el código fuente con el comportamiento lógico de la estructura.

El visualizador busca cerrar esta brecha entre código, modelo abstracto y representación gráfica.

2.2 Usuarios objetivo

Usuario	Descripción
Estudiante	Usa el sistema para comprender estructuras de datos y analizar operaciones paso a paso.
Docente	Usa el sistema como apoyo didáctico en clase y laboratorio.
Monitor académico	Usa el sistema para explicar ejercicios y revisar comportamiento de TAD.
Desarrollador académico	Mantiene y amplía el sistema con nuevos módulos.

2.3 Beneficios esperados

- Favorecer el aprendizaje visual de punteros y memoria dinámica.
- Relacionar la ejecución de código con el cambio de estado de la estructura.
- Apoyar el análisis de complejidad algorítmica.
- Promover buenas prácticas de modularidad, documentación y uso de TAD.
- Servir como base para futuros módulos de estructuras jerárquicas, asociativas y grafos.

2.4 Contexto de uso

La aplicación se usará en sesiones de clase, laboratorios de programación y espacios de estudio autónomo. Debe ejecutarse como aplicación de escritorio en Windows, preferiblemente desde Visual Studio Code, Code::Blocks o un entorno compatible con GCC y Raylib.

---

3. Restricciones técnicas

ID	Restricción
RT-01	El código del núcleo lógico debe escribirse en C estándar.
RT-02	La interfaz gráfica debe desarrollarse con `raylib.h`.
RT-03	No se debe mezclar la lógica del TAD con la lógica gráfica.
RT-04	Los archivos `.h` deben exponer únicamente la API pública de cada TAD.
RT-05	Los archivos `.c` deben encapsular la implementación concreta de cada estructura.
RT-06	Todo módulo, estructura y subrutina pública debe documentarse con Doxygen.
RT-07	Las operaciones deben validar punteros nulos, estructuras vacías y errores de memoria.
RT-08	La aplicación debe poder compilarse con GCC usando Raylib.
RT-09	Los recursos gráficos institucionales deben almacenarse en la carpeta `assets/`.
RT-10	La aplicación debe separar presentación, lógica de dominio y control de interacción.

---

4. Arquitectura general del sistema

4.1 Estilo arquitectónico

Se propone una arquitectura modular en capas, orientada a separación de responsabilidades:

```
+-------------------------------------------------------------+
|                      Capa de Presentación                   |
|              Raylib UI, paneles, botones, dibujo            |
+-------------------------------------------------------------+
|                    Capa de Control Visual                   |
|        Manejo de eventos, selección de estructura, pasos    |
+-------------------------------------------------------------+
|                    Capa de Servicios Didácticos             |
|     Trazas, explicación, complejidad, visor de código        |
+-------------------------------------------------------------+
|                    Capa de Dominio / TAD                    |
|       pila.c, cola.c, cola_prioridad.c, lista.c              |
+-------------------------------------------------------------+
|                    Capa de Infraestructura                  |
|       Raylib, assets, fuentes, compilación, archivos         |
+-------------------------------------------------------------+
```

4.2 Principios de diseño

1. **Separación de responsabilidades:** los TAD no deben depender de Raylib.
2. **Reutilización:** los TAD deben poder usarse en consola, pruebas o interfaz gráfica.
3. **Encapsulamiento:** el usuario del TAD no debe acceder directamente a los nodos internos.
4. **Trazabilidad didáctica:** cada operación debe asociarse a una explicación, código y complejidad.
5. **Extensibilidad:** debe ser posible agregar nuevos módulos, como árboles o grafos, sin reescribir la arquitectura.

---

5. Estructura propuesta de carpetas

```
visualizador-ed/
│
├── assets/
│   ├── logo_utp.png
│   ├── logo_isc.png
│   └── fonts/
│
├── include/
│   ├── pila.h
│   ├── cola.h
│   ├── cola_prioridad.h
│   ├── lista.h
│   ├── ui.h
```

```
│   ├── visualizador.h
│   ├── code_viewer.h
│   ├── algorithm_trace.h
│   └── app_state.h
│
├── src/
│   ├── main.c
│   ├── pila.c
│   ├── cola.c
│   ├── cola_prioridad.c
│   ├── lista.c
│   ├── ui.c
│   ├── visualizador.c
│   ├── code_viewer.c
│   ├── algorithm_trace.c
│   ├── pila_view.c
│   ├── cola_view.c
│   ├── cola_prioridad_view.c
│   └── lista_view.c
│
├── docs/
│   ├── Doxyfile
│   ├── sdd.md
│   └── manual_usuario.md
│
├── tests/
│   ├── test_pila.c
│   ├── test_cola.c
│   ├── test_cola_prioridad.c
│   └── test_lista.c
│
└── Makefile
```

---

6. Modelo de componentes

6.1 Componentes principales

Componente	Responsabilidad
`main.c`	Inicializa la ventana, carga recursos, ejecuta el ciclo principal.
`ui.c` / `ui.h`	Dibuja encabezado, pie de página, estilos institucionales y elementos comunes.
`pila.c` / `pila.h`	Implementa el TAD Pila.
`cola.c` / `cola.h`	Implementa el TAD Cola FIFO.

Componente	Responsabilidad
`cola_prioridad.c` / `cola_prioridad.h`	Implementa el TAD Cola de Prioridad.
`lista.c` / `lista.h`	Implementa el TAD Lista Enlazada.
`pila_view.c`	Dibuja la pila y sus operaciones.
`cola_view.c`	Dibuja la cola FIFO.
`cola_prioridad_view.c`	Dibuja la cola de prioridad con valor y prioridad.
`lista_view.c`	Dibuja la lista enlazada y sus enlaces.
`code_viewer.c`	Muestra fragmentos de código asociados a la operación activa.
`algorithm_trace.c`	Administra mensajes paso a paso de la operación en ejecución.
`app_state.c`	Centraliza el estado global de la aplicación sin mezclarlo con los TAD.

7. Requisitos funcionales

ID	Requisito funcional	Prioridad	Actor
RF-01	El sistema debe permitir seleccionar la estructura de datos a visualizar: pila, cola, cola de prioridad o lista enlazada.	Alta	Usuario
RF-02	El sistema debe mostrar gráficamente los nodos de la estructura seleccionada.	Alta	Usuario
RF-03	El sistema debe permitir insertar valores en la estructura seleccionada.	Alta	Usuario
RF-04	El sistema debe permitir eliminar valores según la semántica de cada estructura.	Alta	Usuario
RF-05	El sistema debe mostrar el código C asociado a la operación seleccionada.	Alta	Usuario
RF-06	El sistema debe mostrar una explicación textual del paso ejecutado.	Alta	Usuario
RF-07	El sistema debe mostrar la complejidad temporal de la operación ejecutada.	Alta	Usuario
RF-08	El sistema debe mostrar etiquetas específicas: `tope`, `frente`, `final`, `cabeza`, `prioridad`, según corresponda.	Alta	Usuario
RF-09	El sistema debe impedir operaciones inválidas, como desapilar una pila vacía o desencolar una cola vacía.	Alta	Usuario

ID	Requisito funcional	Prioridad	Actor
RF-10	El sistema debe permitir limpiar o reiniciar la estructura actual.	Media	Usuario
RF-11	El sistema debe permitir ejecutar operaciones paso a paso.	Alta	Usuario
RF-12	El sistema debe permitir visualizar el estado textual de la estructura.	Media	Usuario
RF-13	El sistema debe permitir buscar un valor en la lista enlazada.	Media	Usuario
RF-14	El sistema debe permitir invertir la lista enlazada.	Media	Usuario
RF-15	El sistema debe permitir calcular operaciones analíticas de lista: promedio, mayor y orden ascendente.	Media	Usuario
RF-16	El sistema debe usar los logos institucionales en el encabezado de la aplicación.	Media	Usuario

8. Requisitos no funcionales

ID	Tipo	Requisito no funcional
RNF-01	Usabilidad	La interfaz debe ser clara, con paneles diferenciados para estructura, código y explicación.
RNF-02	Rendimiento	La aplicación debe mantener una tasa de refresco estable de al menos 30 FPS.
RNF-03	Mantenibilidad	Cada módulo debe estar separado en `.h` y `.c`, evitando archivos monolíticos.
RNF-04	Documentación	Todas las funciones públicas deben estar documentadas con Doxygen.
RNF-05	Portabilidad	El sistema debe compilar con GCC y Raylib en Windows.
RNF-06	Seguridad de memoria	Toda memoria dinámica reservada debe ser liberada.
RNF-07	Robustez	Las funciones deben validar punteros nulos y fallos de memoria.
RNF-08	Legibilidad	El código debe usar nombres descriptivos y estilo consistente.
RNF-09	Extensibilidad	La arquitectura debe permitir agregar árboles y grafos como módulos posteriores.
RNF-10	Consistencia visual	El diseño debe mantener colores institucionales y distribución homogénea.

9. Casos de uso

9.1 Diagrama de casos de uso en PlantUML

```
@startuml
left to right direction
actor Estudiante
actor Docente

rectangle "Visualizador de Estructuras de Datos" {
  usecase "Seleccionar estructura" as UC1
  usecase "Insertar valor" as UC2
  usecase "Eliminar valor" as UC3
  usecase "Visualizar nodos" as UC4
  usecase "Ver código ejecutado" as UC5
  usecase "Ver explicación paso a paso" as UC6
  usecase "Consultar complejidad" as UC7
  usecase "Reiniciar estructura" as UC8
  usecase "Buscar en lista" as UC9
  usecase "Invertir lista" as UC10
}

Estudiante --> UC1
Estudiante --> UC2
Estudiante --> UC3
Estudiante --> UC4
Estudiante --> UC5
Estudiante --> UC6
Estudiante --> UC7
Estudiante --> UC8
Estudiante --> UC9
Estudiante --> UC10

Docente --> UC1
Docente --> UC4
Docente --> UC5
Docente --> UC6
Docente --> UC7
@enduml
```

9.2 Caso de uso CU-01 — Seleccionar estructura

Campo	Descripción
Nombre	Seleccionar estructura
Actor	Estudiante / Docente
Descripción	Permite elegir entre pila, cola, cola de prioridad y lista enlazada.
Precondición	La aplicación está abierta.

Campo	Descripción
Flujo principal	El usuario selecciona una opción del menú lateral. El sistema actualiza el panel gráfico, las operaciones disponibles y el texto introductorio.
Flujo alternativo	Si no hay estructura seleccionada, se muestra una pantalla de bienvenida.
Postcondición	La estructura seleccionada queda activa.

9.3 Caso de uso CU-02 — Insertar valor

Campo	Descripción
Nombre	Insertar valor
Actor	Estudiante
Descripción	Inserta un valor entero en la estructura activa.
Precondición	Debe existir una estructura activa.
Flujo principal	El usuario escribe un valor. Presiona el botón de inserción. El sistema llama al TAD correspondiente. Se actualiza la visualización, el código y la explicación.
Flujo alternativo	Si el valor no es entero, el sistema muestra error de validación.
Postcondición	La estructura queda actualizada.

9.4 Caso de uso CU-03 — Eliminar valor

Campo	Descripción
Nombre	Eliminar valor
Actor	Estudiante
Descripción	Elimina un elemento según la estructura activa.
Precondición	La estructura debe contener al menos un nodo.
Flujo principal	El usuario presiona eliminar. El sistema ejecuta `pop`, `desencolar`, `desencolar prioridad` o eliminación de lista.
Flujo alternativo	Si la estructura está vacía, se informa que la operación no es válida.
Postcondición	La estructura queda actualizada y se muestra el nodo retirado.

9.5 Caso de uso CU-04 — Ver código ejecutado

Campo	Descripción
Nombre	Ver código ejecutado
Actor	Estudiante / Docente

Campo	Descripción
Descripción	Muestra el fragmento de código C correspondiente a la operación seleccionada.
Precondición	Debe existir una operación activa.
Flujo principal	El sistema carga el fragmento desde una tabla interna de cadenas o desde archivos de código.
Postcondición	El estudiante puede relacionar la operación visual con su implementación.

---

10. Contratos de interfaz UI/UX

10.1 Distribución de pantalla propuesta

```
+-------------------------------------------------------------------------------
+
| Logo UTP       Universidad Tecnológica de Pereira / ISC             Logo
ISC   |
+-------------------------------------------------------------------------------
+
| Menú lateral |     Panel de visualización        | Panel de código
ejecutado       |
|              |     de estructura
|                                  |
|              |
|                                  |
|
|-----------------------------------|--------------------------------|
|              |     Panel de explicación          | Panel de complejidad /
estado   |
+-------------------------------------------------------------------------------
+
| Visualizador de Estructuras de Datos Secuenciales —
UTP                         |
+-------------------------------------------------------------------------------
+
```

10.2 Pantallas principales

Pantalla	Descripción
`PANTALLA_INICIO`	Muestra bienvenida, logos y descripción del módulo.
`PANTALLA_PILA`	Visualiza una pila con el tope arriba.
`PANTALLA_COLA`	Visualiza una cola FIFO horizontal, con frente y final.
`PANTALLA_COLA_PRIORIDAD`	Visualiza una cola con valor y prioridad.
`PANTALLA_LISTA`	Visualiza una lista enlazada simple con nodos y flechas.

Pantalla	Descripción
`PANTALLA_AYUDA`	Muestra instrucciones de uso.

10.3 Contratos de eventos

Pantalla	Evento	Acción	Destino
Inicio	Clic en “Pila”	Activar TAD Pila	Pantalla Pila
Inicio	Clic en “Cola”	Activar TAD Cola	Pantalla Cola
Inicio	Clic en “Cola de prioridad”	Activar TAD ColaPrioridad	Pantalla ColaPrioridad
Inicio	Clic en “Lista”	Activar TAD Lista	Pantalla Lista
Pila	Botón Push	Ejecutar `pila_push`	Pila actualizada
Pila	Botón Pop	Ejecutar `pila_pop`	Pila actualizada
Cola	Botón Encolar	Ejecutar `cola_encolar`	Cola actualizada
Cola	Botón Desencolar	Ejecutar `cola_desencolar`	Cola actualizada
ColaPrioridad	Botón Encolar	Ejecutar `cp_encolar`	ColaPrioridad actualizada
ColaPrioridad	Botón Desencolar	Ejecutar `cp_desencolar`	ColaPrioridad actualizada
Lista	Botón Insertar inicio	Ejecutar `lista_insertar_inicio`	Lista actualizada
Lista	Botón Insertar final	Ejecutar `lista_insertar_final`	Lista actualizada
Lista	Botón Buscar	Ejecutar `lista_buscar_posiciones`	Resultado de búsqueda
Lista	Botón Invertir	Ejecutar `lista_invertir`	Lista actualizada

11. Modelo de datos

11.1 TAD Pila

```
typedef struct nodo Nodo;

typedef struct {
    Nodo *tope;
} Pila;
```

Campo	Tipo	Descripción
`tope`	`Nodo *`	Apunta al nodo superior de la pila.

Operaciones públicas:

```
void pila_inicializar(Pila *pila);
bool pila_push(Pila *pila, int valor);
bool pila_pop(Pila *pila, int *valor);
void pila_destruir(Pila *pila);
int pila_contar(const Pila *pila);
bool pila_vacia(const Pila *pila);
int pila_copiar_valores(const Pila *pila, int *destino, int capacidad);
void pila_formatear(const Pila *pila, char *destino, size_t capacidad);
```

11.2 TAD Cola

```
typedef struct nodo Nodo;

typedef struct {
    Nodo *delante;
    Nodo *atras;
} Cola;
```

Campo	Tipo	Descripción
`delante`	`Nodo *`	Apunta al primer nodo de la cola.
`atras`	`Nodo *`	Apunta al último nodo de la cola.

Operaciones públicas:

```
void cola_inicializar(Cola *cola);
bool cola_encolar(Cola *cola, int valor);
bool cola_desencolar(Cola *cola, int *valor);
void cola_vaciar(Cola *cola);
bool cola_vacia(const Cola *cola);
int cola_contar(const Cola *cola);
int cola_copiar_valores(const Cola *cola, int *destino, int capacidad);
void cola_formatear(const Cola *cola, char *destino, size_t capacidad);
```

11.3 TAD Cola de prioridad

```
typedef struct cp_nodo CPNodo;

typedef struct {
    CPNodo *delante;
```

```
    CPNodo *atras;
} ColaPrioridad;
```

Campo	Tipo	Descripción
`delante`	`CPNodo *`	Apunta al primer nodo de la cola física.
`atras`	`CPNodo *`	Apunta al último nodo de la cola física.

Operaciones públicas:

```
void cp_inicializar(ColaPrioridad *cola);
bool cp_encolar(ColaPrioridad *cola, int valor, int prioridad);
bool cp_desencolar(ColaPrioridad *cola, int *valor, int *prioridad);
bool cp_vacia(const ColaPrioridad *cola);
int cp_contar(const ColaPrioridad *cola);
int cp_copiar_items(const ColaPrioridad *cola, int *valores, int
*prioridades, int capacidad);
void cp_formatear(const ColaPrioridad *cola, char *destino, size_t
capacidad);
void cp_vaciar(ColaPrioridad *cola);
```

11.4 TAD Lista enlazada

```
typedef struct nodo Nodo;

typedef struct {
    Nodo *cabeza;
} Lista;
```

Campo	Tipo	Descripción
`cabeza`	`Nodo *`	Apunta al primer nodo de la lista.

Operaciones públicas:

```
void lista_inicializar(Lista *lista);
bool lista_insertar_inicio(Lista *lista, int valor);
bool lista_insertar_final(Lista *lista, int valor);
bool lista_insertar_antes(Lista *lista, int valor, int pos);
bool lista_insertar_despues(Lista *lista, int valor, int pos);
int lista_buscar_posiciones(const Lista *lista, int valor, int *destino, int
capacidad);
bool lista_eliminar_primero(Lista *lista, int valor);
int lista_eliminar_todos(Lista *lista, int valor);
void lista_invertir(Lista *lista);
bool lista_promedio(const Lista *lista, float *resultado);
bool lista_mayor(const Lista *lista, int *resultado);
```

```
bool lista_orden_asc(const Lista *lista);
bool lista_vacia(const Lista *lista);
int lista_contar(const Lista *lista);
int lista_copiar_valores(const Lista *lista, int *destino, int capacidad);
void lista_formatear(const Lista *lista, char *destino, size_t capacidad);
void lista_destruir(Lista *lista);
```

---

12. Estado de la aplicación

12.1 Enumeraciones sugeridas

```
typedef enum {
    ESTRUCTURA_NINGUNA = 0,
    ESTRUCTURA_PILA,
    ESTRUCTURA_COLA,
    ESTRUCTURA_COLA_PRIORIDAD,
    ESTRUCTURA_LISTA
} TipoEstructura;

typedef enum {
    OPERACION_NINGUNA = 0,
    OPERACION_INSERTAR,
    OPERACION_ELIMINAR,
    OPERACION_BUSCAR,
    OPERACION_INVERTIR,
    OPERACION_LIMPIAR
} TipoOperacion;
```

12.2 Estructura de estado global

```
typedef struct {
    TipoEstructura estructura_activa;
    TipoOperacion operacion_activa;

    Pila pila;
    Cola cola;
    ColaPrioridad cola_prioridad;
    Lista lista;

    int valor_entrada;
    int prioridad_entrada;
    int posicion_entrada;

    char mensaje[256];
    char codigo_actual[2048];
```

```
    char complejidad_actual[128];
} AppState;
```

---

13. Reglas de negocio académicas

ID	Regla
RN-01	Una pila debe respetar la política LIFO.
RN-02	Una cola debe respetar la política FIFO.
RN-03	En la cola de prioridad, menor valor numérico representa mayor prioridad.
RN-04	La lista enlazada debe mantener enlaces válidos después de cada inserción o eliminación.
RN-05	Toda operación debe actualizar visualización, explicación, código y complejidad.
RN-06	No se debe permitir `pop` sobre pila vacía.
RN-07	No se debe permitir `desencolar` sobre cola vacía.
RN-08	Toda memoria reservada dinámicamente debe liberarse antes de cerrar la aplicación.
RN-09	La visualización no debe modificar directamente los nodos del TAD.
RN-10	El panel de código debe mostrar código coherente con los TAD suministrados.

---

14. Especificación de algoritmos y complejidad

14.1 Pila — Push

Campo	Descripción
Entrada	Pila y valor entero.
Salida	Pila con un nuevo nodo en el tope.
Complejidad temporal	`O(1)`
Complejidad espacial	`O(1)` por nodo insertado.

Pseudocódigo:

```
crear nuevo nodo
si no hay memoria, retornar falso
nuevo.nro = valor
nuevo.sgte = pila.tope
pila.tope = nuevo
retornar verdadero
```

14.2 Pila — Pop

Campo	Descripción
Entrada	Pila y referencia para guardar valor.
Salida	Valor retirado del tope.
Complejidad temporal	`O(1)`
Complejidad espacial	`O(1)`

Pseudocódigo:

```
si pila vacía, retornar falso
aux = pila.tope
valor = aux.nro
pila.tope = aux.sgte
liberar aux
retornar verdadero
```

14.3 Cola — Encolar

Campo	Descripción
Entrada	Cola y valor entero.
Salida	Cola con nuevo nodo al final.
Complejidad temporal	`O(1)`
Complejidad espacial	`O(1)` por nodo insertado.

14.4 Cola — Desencolar

Campo	Descripción
Entrada	Cola y referencia para guardar valor.
Salida	Valor retirado del frente.
Complejidad temporal	`O(1)`
Complejidad espacial	`O(1)`

14.5 Cola de prioridad — Encolar

Campo	Descripción
Entrada	Cola de prioridad, valor y prioridad.
Salida	Nodo agregado al final físico de la cola.
Complejidad temporal	`O(1)`

Campo	Descripción
Complejidad espacial	`O(1)`

14.6 Cola de prioridad — Desencolar

Campo	Descripción
Entrada	Cola de prioridad.
Salida	Elemento con mayor prioridad efectiva.
Complejidad temporal	`O(n)`
Complejidad espacial	`O(1)`

Pseudocódigo:

```
si cola vacía, retornar falso
actual = delante
min = actual
recorrer todos los nodos
    si actual.prioridad < min.prioridad
        min = actual
eliminar min ajustando enlaces
liberar min
retornar valor y prioridad
```

14.7 Lista — Insertar al inicio

Campo	Descripción
Entrada	Lista y valor entero.
Salida	Lista con nuevo nodo en cabeza.
Complejidad temporal	`O(1)`
Complejidad espacial	`O(1)`

14.8 Lista — Insertar al final

Campo	Descripción
Entrada	Lista y valor entero.
Salida	Lista con nuevo nodo al final.
Complejidad temporal	`O(n)` en lista simple sin puntero final.
Complejidad espacial	`O(1)`

14.9 Lista — Buscar

Campo	Descripción
Entrada	Lista y valor buscado.
Salida	Posiciones donde aparece el valor.
Complejidad temporal	`O(n)`
Complejidad espacial	`O(k)`, donde `k` es la cantidad de coincidencias almacenadas.

---

15. Diseño visual de estructuras

15.1 Pila

Representación vertical con el tope en la parte superior:

```
   TOPE
+--------+
|   30   |
+--------+
|   20   |
+--------+
|   10   |
+--------+
```

Elementos visuales:

- Etiqueta `TOPE`.
- Flecha hacia el nodo superior.
- Resaltado del nodo insertado o eliminado.
- Código `pila_push` o `pila_pop` en el panel derecho.

15.2 Cola FIFO

Representación horizontal:

```
FRENTE                                        FINAL
  ↓                                             ↓
+----+    +----+    +----+    +----+
| 10 | -> | 20 | -> | 30 | -> | 40 |
+----+    +----+    +----+    +----+
```

Elementos visuales:

- Etiqueta `FRENTE`.
- Etiqueta `FINAL`.

- Flechas entre nodos.
- Desencolado desde la izquierda.
- Encolado hacia la derecha.

15.3 Cola de prioridad

Cada nodo debe mostrar valor y prioridad:

```
+-------------+    +-------------+    +-------------+
| valor: 50   | -> | valor: 80   | -> | valor: 20   |
| prioridad:2 |    | prioridad:1 |    | prioridad:3 |
+-------------+    +-------------+    +-------------+
```

Regla visual:

- El nodo con menor número de prioridad se resalta como próximo a salir.
- El panel de explicación debe aclarar que menor prioridad numérica significa mayor prioridad efectiva.

15.4 Lista enlazada simple

```
CABEZA
  ↓
+----+    +----+    +----+    NULL
| 10 | -> | 20 | -> | 30 | -> /
+----+    +----+    +----+
```

Elementos visuales:

- Etiqueta `CABEZA`.
- Flechas entre nodos.
- Indicador `NULL` al final.
- Resaltado de nodo encontrado, insertado o eliminado.

---

16. Visor de código ejecutado

16.1 Propósito

El visor de código debe mostrar un fragmento de código real o adaptado del TAD que corresponde a la operación ejecutada. Su objetivo no es reemplazar el editor de código, sino vincular la acción visual con la implementación.

16.2 Estructura sugerida

```
typedef struct {
    TipoEstructura estructura;
    TipoOperacion operacion;
```

```
    const char *titulo;
    const char *codigo;
    const char *complejidad;
    const char *explicacion;
} CodeSnippet;
```

16.3 Ejemplo de fragmento para pila push

```
static const char *CODIGO_PILA_PUSH =
"bool pila_push(Pila *pila, int valor) {\n"
"    Nodo *aux;\n"
"    if (pila == NULL) {\n"
"        return false;\n"
"    }\n"
"    aux = (Nodo *)malloc(sizeof(Nodo));\n"
"    if (aux == NULL) {\n"
"        return false;\n"
"    }\n"
"    aux->nro = valor;\n"
"    aux->sgte = pila->tope;\n"
"    pila->tope = aux;\n"
"    return true;\n"
"}";
```

---

17. Trazas didácticas

17.1 Objetivo

El sistema debe generar mensajes didácticos que expliquen qué ocurre en cada operación.

17.2 Ejemplo de traza para `pila_push`

Paso	Mensaje
1	Se solicita insertar un nuevo valor en la pila.
2	Se reserva memoria dinámica para un nuevo nodo.
3	El nuevo nodo almacena el valor ingresado.
4	El nuevo nodo apunta al antiguo tope.
5	El puntero `tope` se actualiza para apuntar al nuevo nodo.
6	La operación finaliza en tiempo constante `O(1)`.

17.3 Ejemplo de traza para `cp_desencolar`

Paso	Mensaje
1	Se verifica que la cola de prioridad no esté vacía.
2	Se recorre la lista para encontrar el nodo con menor número de prioridad.
3	Se identifica el nodo de mayor prioridad efectiva.
4	Se ajustan los enlaces para retirar ese nodo.
5	Se libera la memoria del nodo eliminado.
6	La operación tiene complejidad `O(n)` porque debe recorrer la estructura.

---

18. Diagrama de componentes en PlantUML

```
@startuml
skinparam componentStyle rectangle

package "Presentación Raylib" {
  [main.c]
  [ui.c]
  [pila_view.c]
  [cola_view.c]
  [cola_prioridad_view.c]
  [lista_view.c]
}

package "Servicios didácticos" {
  [code_viewer.c]
  [algorithm_trace.c]
  [app_state.c]
}

package "TAD secuenciales" {
  [pila.c]
  [cola.c]
  [cola_prioridad.c]
  [lista.c]
}

[main.c] --> [ui.c]
[main.c] --> [app_state.c]
[pila_view.c] --> [pila.c]
[cola_view.c] --> [cola.c]
[cola_prioridad_view.c] --> [cola_prioridad.c]
[lista_view.c] --> [lista.c]
[pila_view.c] --> [code_viewer.c]
[cola_view.c] --> [code_viewer.c]
```

```
[cola_prioridad_view.c] --> [code_viewer.c]
[lista_view.c] --> [code_viewer.c]
[code_viewer.c] --> [algorithm_trace.c]
@enduml
```

---

19. Diagrama de paquetes en PlantUML

```
@startuml
package "app" {
  class AppState
  class MainLoop
}

package "ui" {
  class UIContext
  class Header
  class Footer
  class Menu
}

package "views" {
  class PilaView
  class ColaView
  class ColaPrioridadView
  class ListaView
}

package "domain" {
  class Pila
  class Cola
  class ColaPrioridad
  class Lista
}

package "didactic" {
  class CodeViewer
  class AlgorithmTrace
  class ComplexityPanel
}

app --> ui
app --> views
views --> domain
views --> didactic
@enduml
```

---

20. Diagrama de clases lógico

```
@startuml
class AppState {
  TipoEstructura estructura_activa
  TipoOperacion operacion_activa
  Pila pila
  Cola cola
  ColaPrioridad cola_prioridad
  Lista lista
  int valor_entrada
  int prioridad_entrada
  char mensaje[256]
}

class Pila {
  Nodo *tope
  pila_push()
  pila_pop()
  pila_contar()
}

class Cola {
  Nodo *delante
  Nodo *atras
  cola_encolar()
  cola_desencolar()
}

class ColaPrioridad {
  CPNodo *delante
  CPNodo *atras
  cp_encolar()
  cp_desencolar()
}

class Lista {
  Nodo *cabeza
  lista_insertar_inicio()
  lista_insertar_final()
  lista_buscar_posiciones()
}

class CodeViewer {
  cargar_codigo()
  dibujar_codigo()
}

class AlgorithmTrace {
  agregar_paso()
```

```
  limpiar_traza()
  dibujar_traza()
}

AppState *-- Pila
AppState *-- Cola
AppState *-- ColaPrioridad
AppState *-- Lista
AppState --> CodeViewer
AppState --> AlgorithmTrace
@enduml
```

---

21. Diagrama de secuencia — operación Push

```
@startuml
actor Usuario
participant "UI Raylib" as UI
participant "AppState" as State
participant "TAD Pila" as Pila
participant "CodeViewer" as Code
participant "AlgorithmTrace" as Trace

Usuario -> UI: Ingresa valor y presiona Push
UI -> State: leer valor_entrada
UI -> Pila: pila_push(&pila, valor)
Pila --> UI: true/false
UI -> Trace: generar pasos de push
UI -> Code: cargar código pila_push
UI -> UI: redibujar pila, código y explicación
@enduml
```

---

22. Diagrama de secuencia — desencolar cola de prioridad

```
@startuml
actor Usuario
participant "UI Raylib" as UI
participant "AppState" as State
participant "TAD ColaPrioridad" as CP
participant "CodeViewer" as Code
participant "AlgorithmTrace" as Trace

Usuario -> UI: Presiona Desencolar
UI -> CP: cp_desencolar(&cola_prioridad, &valor, &prioridad)
CP -> CP: recorrer nodos
CP -> CP: identificar menor prioridad numérica
```

```
CP -> CP: ajustar enlaces
CP -> CP: liberar nodo
CP --> UI: valor, prioridad, estado
UI -> Trace: mostrar pasos de recorrido y eliminación
UI -> Code: cargar código cp_desencolar
UI -> UI: redibujar estructura
@enduml
```

---

23. Criterios de aceptación

ID	Criterio
CA-01	La aplicación permite seleccionar entre pila, cola, cola de prioridad y lista.
CA-02	Cada estructura se dibuja correctamente después de cada operación.
CA-03	El sistema usa los TAD suministrados y no duplica la lógica de estructuras dentro de la capa gráfica.
CA-04	La pila respeta LIFO.
CA-05	La cola respeta FIFO.
CA-06	La cola de prioridad extrae el elemento con menor número de prioridad.
CA-07	La lista permite insertar, buscar, eliminar e invertir.
CA-08	El panel de código muestra el fragmento asociado a la operación activa.
CA-09	El panel de explicación muestra pasos comprensibles para el estudiante.
CA-10	El sistema muestra la complejidad de cada operación.
CA-11	Las operaciones inválidas se informan sin cerrar la aplicación.
CA-12	El cierre de la aplicación libera la memoria dinámica de las estructuras.
CA-13	La documentación Doxygen puede generarse sin errores críticos.
CA-14	La aplicación compila con GCC y Raylib.

---

24. Plan de pruebas

ID	Caso de prueba	Entrada	Resultado esperado
PT-01	Push en pila vacía	10	La pila muestra 10 como tope.
PT-02	Push múltiple	10, 20, 30	El tope es 30.
PT-03	Pop	pila con 10, 20, 30	Retira 30.
PT-04	Pop en pila vacía	pila vacía	Mensaje de error controlado.

ID	Caso de prueba	Entrada	Resultado esperado
PT-05	Encolar en cola vacía	10	Frente y final apuntan a 10.
PT-06	Encolar múltiple	10, 20, 30	Frente 10, final 30.
PT-07	Desencolar	cola con 10, 20, 30	Retira 10.
PT-08	Desencolar cola vacía	cola vacía	Mensaje de error controlado.
PT-09	Encolar prioridad	(50,2), (80,1), (20,3)	Se dibujan los tres nodos con prioridad.
PT-10	Desencolar prioridad	(50,2), (80,1), (20,3)	Retira 80 con prioridad 1.
PT-11	Insertar inicio lista	10	Cabeza apunta a 10.
PT-12	Insertar final lista	10, 20	Lista 10 -> 20.
PT-13	Buscar en lista	buscar 20	Se resalta el nodo con 20.
PT-14	Eliminar primero lista	eliminar 10	Se ajusta cabeza o enlace anterior.
PT-15	Invertir lista	10 -> 20 -> 30	Resultado 30 -> 20 -> 10.
PT-16	Cierre de aplicación	salir	Se libera toda la memoria.

---

25. Estándar de documentación Doxygen

25.1 Encabezado de archivo

Cada archivo debe iniciar con:

```
/**
 * @file nombre_archivo.c
 * @brief Descripción breve del módulo.
 *
 * Descripción extendida del propósito del archivo, estructura que
implementa,
 * responsabilidades principales y restricciones de uso.
 *
 * @author Francisco Alejandro Medina Aguirre
 * @date 2026
 */
```

25.2 Documentación de estructuras

```
/**
 * @struct NombreEstructura
 * @brief Descripción breve de la estructura.
 *
 * Descripción detallada de su papel dentro del TAD o la interfaz.
 */
```

```c
typedef struct {
    int campo; /**< Descripción del campo. */
} NombreEstructura;
```

25.3 Documentación de funciones

```c
/**
 * @brief Descripción breve de la función.
 *
 * Descripción detallada de lo que hace, condiciones especiales,
 * manejo de errores y efectos sobre la estructura.
 *
 * @param[in] parametro_entrada Descripción del parámetro de entrada.
 * @param[out] parametro_salida Descripción del parámetro de salida.
 * @param[in,out] estructura Descripción del parámetro modificado.
 *
 * @return Descripción del valor retornado.
 *
 * @pre Condición previa requerida.
 * @post Estado esperado después de ejecutar la función.
 * @note Complejidad temporal: O(...).
 */
```

25.4 Ejemplo aplicado

```c
/**
 * @brief Inserta un valor entero en el tope de la pila.
 *
 * Reserva memoria dinámica para un nuevo nodo, almacena el valor recibido
 * y actualiza el puntero al tope de la pila.
 *
 * @param[in,out] pila Puntero a la pila destino.
 * @param[in] valor Valor entero que se desea insertar.
 *
 * @return true si la inserción fue exitosa.
 * @return false si la pila es NULL o no hay memoria disponible.
 *
 * @pre La pila debe haber sido inicializada con pila_inicializar().
 * @post El tamaño lógico de la pila aumenta en una unidad.
 * @note Complejidad temporal: O(1).
 */
bool pila_push(Pila *pila, int valor);
```

---

26. Integración con los archivos suministrados

26.1 Archivos TAD

Los archivos `pila.h`, `cola.h`, `cola_prioridad.h` y `lista.h` ya presentan una separación adecuada entre interfaz pública e implementación. La aplicación gráfica debe consumir estas APIs sin acceder directamente a los nodos internos.

26.2 Archivos gráficos existentes

Los archivos `pilas_raylib.c`, `colas_raylib.c`, `colas_prioridad_raylib.c` y `listas_raylib.c` se deben reorganizar progresivamente hacia una estructura más homogénea:

Archivo actual	Evolución sugerida
`pilas_raylib.c`	`pila_view.c`
`colas_raylib.c`	`cola_view.c`
`colas_prioridad_raylib.c`	`cola_prioridad_view.c`
`listas_raylib.c`	`lista_view.c`

26.3 UI institucional

El módulo `ui.c` debe encargarse exclusivamente de:

- Cargar logos institucionales.
- Dibujar encabezado.
- Dibujar pie de página.
- Definir colores institucionales.
- Mantener consistencia visual general.

No debe contener lógica de TAD ni lógica de operaciones.

---

27. Estrategia de implementación incremental

Fase 1 — Normalización del proyecto

- Organizar carpetas `src`, `include`, `assets`, `docs` y `tests`.
- Verificar compilación base con Raylib.
- Ajustar rutas de logos.
- Crear `Makefile`.

Fase 2 — Integración de TAD

- Compilar y probar `pila.c`, `cola.c`, `cola_prioridad.c` y `lista.c` sin interfaz gráfica.
- Crear pruebas mínimas de consola.
- Verificar memoria y retornos booleanos.

Fase 3 — Interfaz principal

- Crear menú lateral.
- Crear panel gráfico.
- Crear panel de código.
- Crear panel de explicación.
- Crear panel de complejidad.

Fase 4 — Visualización de pila y cola

- Implementar `pila_view.c`.
- Implementar `cola_view.c`.
- Asociar operaciones con fragmentos de código y trazas.

Fase 5 — Visualización de cola de prioridad y lista

- Implementar `cola_prioridad_view.c`.
- Implementar `lista_view.c`.
- Agregar resaltado de nodos encontrados o eliminados.

Fase 6 — Documentación y pruebas

- Completar comentarios Doxygen.
- Generar documentación HTML.
- Ejecutar pruebas funcionales.
- Preparar guía de laboratorio para estudiantes.

---

28. Comando de compilación sugerido

En Windows con MSYS2 UCRT64 y Raylib instalado:

```
gcc src/main.c src/ui.c src/pila.c src/cola.c src/cola_prioridad.c src/
lista.c
    src/pila_view.c src/cola_view.c src/cola_prioridad_view.c src/
lista_view.c
    src/code_viewer.c src/algorithm_trace.c src/app_state.c
    -Iinclude -o visualizador.exe -lraylib -lopengl32 -lgdi32 -lwinmm
```

Con `Makefile`, el objetivo ideal sería:

```
make run
```

---

29. Riesgos técnicos

Riesgo	Impacto	Mitigación
Mezclar lógica gráfica con TAD	Alto	Mantener los TAD independientes de Raylib.
Fugas de memoria	Alto	Usar funciones destruir/vaciar en cierre y reinicio.
Código duplicado en vistas	Medio	Crear funciones comunes para dibujar tarjetas, botones y paneles.
Dificultad para mostrar código largo	Medio	Usar scroll vertical en el panel de código.
Incompatibilidad de fuentes o tildes	Medio	Cargar fuente TrueType compatible desde Windows o assets.
Exceso de nodos en pantalla	Medio	Implementar scroll o escala visual.
Operaciones inválidas	Medio	Validar estado antes de cada operación.

---

30. Recomendaciones pedagógicas de uso

1. Iniciar cada clase mostrando el comportamiento abstracto de la estructura.
2. Ejecutar una operación visualmente antes de mostrar el código.
3. Pedir al estudiante que prediga el estado final antes de presionar el botón.
4. Relacionar cada operación con su complejidad.
5. Usar el panel de código para explicar punteros y memoria dinámica.
6. Complementar el visualizador con ejercicios escritos y prácticas de laboratorio.

---

31. Definición de terminado

El módulo 1 se considerará terminado cuando:

- Compile correctamente con Raylib.
- Permita operar pila, cola, cola de prioridad y lista.
- Muestre visualmente cada estructura.
- Muestre código asociado a cada operación.
- Muestre explicación y complejidad.
- Use los TAD suministrados sin duplicar su lógica.
- Libere memoria correctamente al cerrar.
- Tenga documentación Doxygen en archivos públicos y funciones principales.
- Cuente con una guía breve de uso para estudiantes.

---

32. Referencias base para el desarrollo académico

- Aho, A. V., Hopcroft, J. E., & Ullman, J. D. Estructuras de datos y algoritmos.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. Introduction to Algorithms.

• Kernighan, B. W., & Ritchie, D. M. The C Programming Language.
• Villalobos, J. A. Diseño y manejo de estructuras de datos en C.
• Material del curso de Estructuras de Datos de la Universidad Tecnológica de Pereira.